1. Project and need

XGBoost: separating early-stopping validation from final evaluation

CSCI 566 | Optional open-source contribution report

I have used XGBoost for SDSS photometric-redshift regression, alongside a PyTorch multilayer perceptron. This makes the distinction between fitting a model, selecting its settings, and evaluating its predictions directly relevant to my work. The XGBoost version used in that earlier project is not recorded here. The contribution in this report uses a public classification example, not SDSS observations or private research data.

XGBoost is an open-source gradient-boosted-tree library. Its scikit-learn interface is documented in the official repository [1]. This work is based on commit 293cad1a89ab749e3266e190c68118af2db09982. The affected file is doc/python/sklearn_estimator.rst [2].

The concrete need. In the original introductory example, the variables named X_test and y_test are supplied to eval_set for early stopping. In the original cross-validation helper, the outer held-out fold is used both to choose the stopping iteration and to calculate test_score. The examples execute, but these data roles can lead readers to interpret a model-selection score as an independent final evaluation.

Early stopping consults validation labels to decide how many boosting rounds to retain. Even though those labels are not used to calculate the training gradients, they still influence the selected predictor. A score on that same data is therefore not independent of selection. A two-way training/validation split is sufficient when only fitting and selecting a model; a separate untouched test set is needed when an independent final assessment is also required.

Scope and deliverables. The upstream-facing change is documentation only: one RST file, with 44 insertions and 23 deletions. It changes neither the training algorithm nor the public API. The accompanying contribution copy contains the original and revised document, xgboost-early-stopping.patch, reproducibility scripts, recorded reference-run evidence, and a rendered HTML preview. No upstream pull request or course submission is claimed by this report.

2. New work

The revised guide makes the data roles explicit in both the introductory example and the cross-validation example.

Introductory example. The first split reserves 20% for the final test set. A second split takes 25% of the remaining development data for validation, producing approximately 60% training, 20% validation, and 20% test data overall. Both splits use stratification and random_state=94. The fit call receives only the training data and the validation eval_set; the test set is scored afterward. These proportions are illustrative rather than a general prescription.

Callback and cross-validation examples. The callback example uses the same X_valid and y_valid variables. In five-fold stratified cross-validation, each outer training partition becomes a development partition. The helper then reserves 20% of that partition for early stopping. The outer test fold is used only for scoring. This is the central contribution excerpt:

```
def fit_and_score(estimator, X_dev, X_test, y_dev, y_test):
    """Select the stopping iteration without using the outer test fold."""
    X_train, X_valid, y_train, y_valid = train_test_split(
        X_dev, y_dev, test_size=0.2, stratify=y_dev, random_state=94
    )
    estimator.fit(X_train, y_train, eval_set=[(X_valid, y_valid)])

    train_score = estimator.score(X_train, y_train)
    test_score = estimator.score(X_test, y_test)
    return estimator, train_score, test_score
```

The outer loop clones the estimator for each fold. Its template is renamed from clf to estimator, so the later native-booster example still refers to the fitted introductory clf. An unused cross_validate import is removed. The explanatory text also clarifies that different folds may select different tree counts while evaluating the same training procedure.

Relation to existing work. Issue #7782 discusses dynamic eval_set and internal splitting [3]; this patch retains explicit user-controlled splits instead of adding that functionality. PR #8948 introduced the guide [4], while PR #9304 clarified best_iteration and inference [5]. Those changes do not address the separation of selection and scoring data here. A targeted review of related history and open/closed issues found no identical fix, but the search was not exhaustive and does not establish maintainer approval.

3. Verification

My local verification. I completed local verification and confirmed it passed; the platform and command details for my local run are not recorded here. My confirmation does not establish that my environment or numerical outputs match the reference run below.

Recorded AI-assisted reference run. The saved evidence uses Python 3.12.14, the XGBoost CPU 3.4.1 wheel, scikit-learn 1.8.0, NumPy 2.3.5, and SciPy 1.17.0. The following observations are from evidence/assistant_run/results.json, not a transcription of my local results. The helper executes code extracted directly from the RST rather than a separately maintained copy of the examples.

Expected behavior and observed results

- **Execution:** The original overview and CV examples should run, and all four revised blocks should run sequentially. Both checks passed; the revised run recorded seven fits and eleven scores.

- **Data separation:** Training, validation, and test row identities should be pairwise disjoint. The introductory split contained 341/114/114 rows. Four CV folds contained 364/91/114 rows and the fifth contained 364/92/113 rows. All separation assertions passed. The callback reused the introductory training and validation partitions.

- **Baseline contrast:** The original CV helper should reveal that the early-stopping set and scored test fold are the same. Their overlaps were 114, 114, 114, 114, and 113 rows: the entire scored fold each time.

- **Selection independence:** With outer-fold membership fixed, flipping only test labels should leave the revised fitted model unchanged. The serialized boosters were byte-identical and both selected zero-based iteration 11. Accuracy changed from 0.95614 to 0.04386, as expected when evaluation labels change. In the original helper, the same perturbation changed the selected iteration from 15 to 0, demonstrating sensitivity to those labels.

- **Artifact checks:** The fixed upstream revision was restored and the patch applied successfully. Patch-format checks passed. An isolated Sphinx 9.1.1 preview built successfully; it uses marked external-reference placeholders and is not a full upstream documentation build.

Reproduction from the contribution copy

```
python -m pip install -r requirements.txt
python scripts/verify_examples.py
python scripts/restore_repo.py restored-xgboost
git -C restored-xgboost diff --check
```

The dependency file uses standard xgboost==3.4.1; the recorded Linux reference run used xgboost-cpu==3.4.1. The verifier should print status PASS and save new results under evidence/rerun/. No source compilation, GPU testing, full repository test suite, full upstream documentation build, or public CI run is claimed. Accuracy is an observed value, not evidence that this documentation change improves prediction quality.

4. Explanation

The design applies the training/validation/test and model-selection concepts in the CSCI 566 review material, pages 4-9 [6]. Training data fit tree parameters. Validation data select the stopping iteration. Test data assess the resulting predictor after selection. The important boundary is whether data influence a decision, not whether a variable happens to be named "test."

For each outer CV fold, the inner validation set must come from that fold's development partition. Taking it from the outer test fold would allow test labels to influence selection. Different stopping iterations across folds are expected: CV evaluates a procedure applied to different samples, not a requirement that every fitted model have the same number of trees. This example fixes hyperparameters. If outer CV scores are later used to select hyperparameters, those scores also become model-selection evidence; an additional held-out test set or nested CV is needed for independent final evaluation.

The label-perturbation check directly probes that boundary. It keeps fold membership fixed so that changing test labels cannot alter the stratified split itself. The unchanged revised model supports the narrow claim that those test labels do not affect this helper's fitting and early stopping. It is not a proof against all forms of leakage, such as preprocessing fitted on the entire dataset. The explicit split also avoids introducing estimator-internal splitting complexities discussed in issue #7782 [3].

Contribution and AI assistance. AI assisted issue and history research, patch drafting, verification-script drafting, the recorded reference experiment, and report preparation. I supplied the personal-use context and confirmed that local verification passed. The numerical reference results above are AI-assisted evidence. This report does not claim that I independently authored every line, inspected every change, or reproduced those exact numbers on my machine.

Sources and contribution artifacts

[1] Official repository: https://github.com/dmlc/xgboost

[2] Pinned source: https://github.com/dmlc/xgboost/blob/293cad1a89ab749e3266e190c68118af2db09982/doc/python/sklearn_estimator.rst

[3] Related feature request and design discussion: https://github.com/dmlc/xgboost/issues/7782

[4] Original guide: https://github.com/dmlc/xgboost/pull/8948

[5] Inference clarification: https://github.com/dmlc/xgboost/pull/9304

[6] CSCI 566 review deck, pp. 4-9: https://drive.google.com/file/d/1NqEtHTPwUhW7UTBlBg1XC3508cKzlZwD/view

Contribution repository: https://github.com/tcf2021/csci566-xgboost-early-stopping

The accompanying contribution archive contains xgboost-early-stopping.patch, scripts/verify_examples.py, evidence/assistant_run/results.json, and review/PRIOR_WORK.md. The complete patch and recorded evidence are provided in the accompanying contribution archive and at the repository URL above.